

SALDO

Documentacion funcional y plan de correcciones Android

Documento preparado para continuar el trabajo cuando se agote el contexto de la sesion.

Fecha: 2026-10-02 | Version observada: 1.0.8C

Alcance: problemas de solapamiento de interfaz movil, convivencia entre navegacion inferior, boton de alta y recurrentes, y mejora de notificaciones Android. Incluye el mapa funcional de modulos y funciones sin codigo fuente.

1. Resumen ejecutivo

La aplicacion funciona y el frontend ya se carga correctamente en Android. Las evidencias aportadas muestran un problema posterior: la interfaz movil no reserva de forma consistente el espacio de las zonas del sistema y de los controles fijos. Como consecuencia, el contenido puede quedar debajo de la barra superior, el boton flotante puede invadir la navegacion inferior y la franja de gastos recurrentes puede superponerse con ambos elementos.

Las notificaciones se emiten, pero Android utiliza un icono generico y el texto expandido no comunica de forma completa el evento. Esto apunta a una configuracion incompleta del canal/icono de notificacion y a mensajes que no separan claramente titulo, resumen y detalle.

Prioridad recomendada: primero corregir el sistema de capas y espacios del layout movil; despues normalizar el componente de recurrentes; por ultimo mejorar iconografia, canales y contenido de notificaciones.

2. Evidencia visual y problemas registrados

Area	Comportamiento observado	Resultado esperado
P1 - Area superior	La pantalla de bienvenida comienza debajo de una zona superior que puede invadir el contenido. El boton final queda pegado a la navegacion del sistema.	Reservar inset superior real y un margen inferior que contemple boton, barra de app y barra del sistema.
P2 - Navegacion inferior	La barra inferior y el FAB de nuevo gasto comparten espacio vertical. En Reportes, el FAB queda encima del bloque de recurrentes.	Definir una geometria unica de zonas fijas y calcular la posicion del FAB y de recurrentes desde esa geometria.
P3 - Recurrentes	La franja "Recurrentes de hoy" aparece sobre el contenido y puede chocar con el menu inferior y el FAB.	Convertirla en bloque de flujo o darle un carril fijo independiente; nunca usar una posicion fija sin reservar espacio equivalente.
P4 - Notificaciones	La notificacion muestra icono generico y al expandirse no ofrece el contexto completo de lo ocurrido.	Usar icono monocromo de notificacion, canal estable y mensajes con titulo, resumen y detalle legibles.

3. Diagnostico tecnico funcional

El CSS movil actualmente usa varios elementos con posicion fija y valores independientes: navegacion inferior, barra de recurrentes, boton flotante y relleno del contenido. Aunque existe soporte para safe area, los valores no forman un modelo unico. El contenido puede reservar 160 px abajo, mientras que las capas superpuestas se posicionan con alturas distintas. Esa falta de una medida comun explica que el resultado dependa de la pantalla, densidad, barra de gestos y pagina actual.

La pagina de bienvenida utiliza un modal de tipo bottom sheet. En la captura, el boton de accion queda muy cerca de la barra de navegacion del sistema. El problema no es solo visual: el area tactil inferior puede quedar parcialmente inaccesible.

La barra de recurrentes se renderiza desde el modulo de recurrentes y tambien existe un widget de gastos recurrentes dentro de otras vistas. Debe diferenciarse claramente entre: bloque en el flujo normal, panel lateral de escritorio, widget contextual y barra movil. Si todos comparten reglas de posicionamiento, se producen colisiones.

4. Plan de correccion de interfaz movil

Paso	Accion funcional	Criterio de aceptacion
1. Modelo de zonas	Definir conceptualmente cuatro zonas: inset superior, contenido desplazable, carril de acciones (recurrentes/FAB) e inset inferior con navegacion.	Ningun elemento fijo debe ocupar el mismo carril que otro.
2. Navegacion inferior	Mantener una altura estable que incluya el inset inferior. El contenido debe tener padding inferior equivalente a la altura completa de la barra.	Ultima tarjeta y botones siempre visibles y pulsables.

Paso	Accion funcional	Criterio de aceptacion
3. FAB	Ubicarlo por encima de la navegacion con una separacion constante. En vistas con recurrentes visibles, el FAB debe desplazarse o compartir un carril coordinado.	No tapa etiquetas, tarjetas ni controles de recurrentes.
4. Recurrentes	En movil preferir un bloque dentro del scroll. Si se mantiene fijo, reservar un carril exclusivo y ocultarlo cuando no haya pendientes.	No invade navegacion ni FAB.
5. Modal de bienvenida	Aumentar el espacio inferior del bottom sheet y asegurar que el boton final respete el inset del sistema.	El boton completo queda dentro del area util y responde al tacto.
6. Pruebas	Probar pantalla pequena, pantalla alta, gestos y tres botones del sistema; revisar Inicio, Reportes, Movimientos y Configuracion.	La solucion no depende de una sola captura o dispositivo.

5. Notificaciones Android

El modulo de notificaciones verifica si el dispositivo es Android, obtiene la API global de Tauri, solicita permiso, consulta el estado y envia notificaciones. La funcionalidad basica ya opera. La mejora pendiente es de presentacion y configuracion nativa.

Aspecto	Comportamiento actual/pendiente	Resultado esperado
Identidad visual	El icono mostrado por Android es generico. Se necesita un recurso de notificacion monocromo, simple y con contraste; no debe reutilizarse sin adaptar el icono de launcher.	El panel muestra la identidad de Saldo en lugar del icono informativo generico.
Canal	Android conserva configuracion por canal. El canal debe tener nombre legible, importancia adecuada y ser consistente entre versiones.	El usuario puede reconocer que pertenece a Saldo y controlar su comportamiento.
Titulo	Usar titulos cortos y especificos: "Presupuesto bajo", "Recurrentes pendientes", "Actualizacion disponible".	El titulo permite entender el evento sin abrir la notificacion.
Detalle expandido	El cuerpo debe incluir que ocurrio, monto o cantidad relevante, ciclo afectado y accion sugerida cuando exista.	Al expandir, la informacion es completa y no depende de contexto externo.
Prueba	Probar notificacion colapsada y expandida, modo oscuro, permiso denegado, canal silenciado y diferentes fabricantes.	El comportamiento es consistente y los errores quedan registrados.

6. Mapa funcional de modulos

La fuente modular se carga en orden desde index.html. Las funciones se apoyan en un estado global persistido en localStorage y cada modulo agrega una capacidad aislada. El orden de carga es importante: constantes, utilidades, estado, ciclos, categorias, ingresos, recurrentes, proyeccion, gastos, ahorros, colectivo, estadisticas, respaldo, interfaz, notificaciones, eventos y arranque.

Modulo	Responsabilidad	Como funciona
00-logs.js	Diagnostico y observabilidad	Inicializa el registro, captura errores globales y promesas rechazadas, persiste eventos localmente y trata de escribirlos en Documentos/Saldo/Logs. Tambien ofrece exportacion, limpieza y envoltura manual para diagnostico.
01-constantes.js	Modelo base	Define claves de almacenamiento, version visible, paleta, iconos, categorias por defecto, estado inicial y clonado compatible con WebView.
02-utilidades.js	Servicios pequenos	Calcula fechas efectivas, dias del mes, escapa HTML, muestra avisos temporales y formatea notas de actualizacion.
03-estado.js	Persistencia	Carga, migra, completa y guarda el estado de la aplicacion. Mantiene variables de trabajo de edicion y seleccion.
04-ciclos.js	Ciclo de presupuesto	Calcula inicio, fin, clave y proximo ciclo; determina si un gasto pertenece al ciclo actual y cierra ciclos completados.
05-categorias.js	Categorias	Busca categorias, completa selectores, construye selector de colores e iconos, abre el editor y renderiza tarjetas/resumen.
06-ingresos.js	Ingresos	Calcula ingreso base y recurrente, ocurrencias, totales confirmados, ingreso actual, pendientes y aplica ingresos aprobados.
07-recurrentes.js	Plantillas recurrentes	Calcula vencimientos y estados, lista pendientes, confirma o pospone, abre editores y renderiza barra, widget y confirmaciones.
08-proyeccion.js	Proyeccion	Normaliza plantillas, genera ocurrencias futuras, calcula desglose, gasto recurrente pendiente, recomendacion y detalle proyectado.
09-gastos.js	Gastos	Filtra gastos del ciclo, calcula presupuesto restante, agrupa por categoria, construye actividad, edita y renderiza movimientos.
10-ahorros.js	Ahorros	Gestiona fecha del sistema, apertura de modal, sugerencia de metas y renderizado de saldo, movimientos y metas.
11-colectivo.js	Compras colectivas	Administra borrador, personas, liquidacion, transferencias, guardado, renderizado, comprobante e inicio de nuevo evento.
12-estadisticas.js	Estadisticas e informes	Renderiza estadisticas, reportes, informe de gastos con estados ABONADO, consejo de pago, historial y vista imprimible.
13-respaldo.js	Respaldo y actualizacion	Genera exportacion compatible con Excel, importa respaldo, valida datos y consulta actualizaciones disponibles.
14-interfaz.js	Navegacion y render	Abre/cierra modales, cambia de vista, renderiza panel, configuracion y el renderizado general de la aplicacion.
15-eventos.js	Interaccion	Conecta clics delegados, teclado, formularios, cambios, acciones de gastos, ingresos, categorias, ahorros, colectivos, respaldo, modo oscuro y logs.
16-iniciar.js	Arranque	Registra el inicio, ejecuta el primer renderizado y programa la verificacion de actualizaciones fuera de Android.
17-notificaciones.js	Android	Detecta Android, consulta API y permisos, activa notificaciones, envia eventos, lanza prueba y actualiza controles.

7. Inventario de funciones por modulo

El siguiente inventario permite retomar el desarrollo sin releer todo el proyecto. Las funciones se agrupan por responsabilidad; no se incluye código.

02-utilidades.js

fechaEfectiva | diasDelMes | escaparHtml | mostrarAviso | fechaDeHoy | claveDeHoy | formatearNotasActualizacion

03-estado.js

cargarEstado | guardarEstado

04-ciclos.js

obtenerInicioCiclo | obtenerFinCiclo | claveFechaCiclo | esCicloActual | proximaFechaCiclo | cerrarCiclosCompletados

05-categorias.js

obtenerCategoria | completarSelectorDeCategorias | completarCategoriaEdicionGasto | construirSelectorDeColor | construirSelectorDelcono | abrirEditorDeCategoria | renderizarCategorias

06-ingresos.js

ingresoBase | plantillasIngresoRecurrente | fechasOcurrienciaRecurrente | totalIngresoRecurrenteConfirmado | ingresoActual | totalIngresoRecurrenteMensual | ingresoRecurrentePendiente | ingresoRecurrenteAprobado | aplicarIngresoPendiente | razonDelIngreso

07-recurrentes.js

fechaVencimientoMensual | claveConfirmacionRecurrente | formatearFrecuenciaTexto | plantillasRecurrentesDeHoy | recurrentesPendientes | plantillasRecurrentesMensuales | plantillasRecurrentesMensualesPendientes | recurrentesMensualesPendientes | confirmarRecurrente | obtenerEtiquetaEstadoRecurrente | proximaFechaRecurrente | actualizarAyudaEdicionRecurrente | abrirEditorRecurrente | renderizarTodosLosRecurrentes | renderizarRecurrentesBarraLateral | renderizarWidgetGastosRecurrentes | renderizarConfirmacionRecurrentes | renderizarAvisoIngresoAprobado

08-proyeccion.js

calcularProyeccionMensual | contarDiasEnRango | interpretarFechaProyeccion | normalizarRecurrenteParaProyeccion | normalizarEstadoProyeccion | estadoActualProyeccion | fechasRecurrentesEnRango | ocurrenciasProyectadas | desgloseProyeccion | totalGastosRecurrentesPendientes | calcularProyeccion | valorProyeccionMensual | calcularRecomendacionMeta | renderizarDetalleProyeccion

09-gastos.js

abrirGastosDeCategoria | gastosDelCicloActual | gastosConfirmados | calcularPresupuestoRestante | agruparPorCategoria | movimientosDeActividad | filaDeGasto | etiquetaModificacionGasto | obtenerRazonEdicionGasto | abrirEditorDeGasto | renderizarGastos | renderizarWidgetGastosRecurrentes

10-ahorros.js

hoyDelSistemaISO | abrirModalDeAhorro | actualizarSugerenciaMetaDeAhorro | renderizarMetasDeAhorro | renderizarAhorros

11-colectivo.js

borradorColectivo | personasIncluidasColectivo | calcularLiquidacionColectiva | agruparTransferenciasColectivas | guardarBorradorColectivo | renderizarColectivo | imprimirComprobanteColectivo | reiniciarColectivo

12-estadisticas.js

renderizarEstadisticas | renderizarReportes | renderizarInformeDeGastos | renderizarConsejoDePago | marcadoHistorialIngresos | renderizarPaginaHistorialIngresos

13-respaldo.js

tablaDeRespaldo | exportarRespaldo | importarRespaldo | verificarActualizacion

14-interfaz.js

abrirModal | cerrarModal | navegar | renderizarPanel | renderizarConfiguracion | actualizarCiclo | renderizar

17-notificaciones.js

esAndroid | apiNotificacionesTauri | notificacionesAndroidDisponibles | permisoNotificacionesAndroid | solicitarPermisoNotificacionesAndroid | notificacionesAndroidActivadas | enviarNotificacionSistema | actualizarControlNotificacionesAndroid | activarNotificacionesAndroid | enviarNotificacionAndroid | lanzarNotificacionPruebaAndroid | revisarNotificacionesAndroid

8. Flujo de arranque y dependencias

Al abrir la aplicacion, el documento HTML carga los modulos en orden. El sistema de logs debe cargarse antes que el resto para registrar fallos. Las constantes crean el modelo base; estado lee localStorage; interfaz y eventos dependen de que existan todos los elementos HTML; el ultimo modulo ejecuta el primer renderizado. Si un script falla antes de completar, la interfaz visual puede quedar visible pero sin eventos o sin datos iniciales.

El preparador de frontend copia index.html, estilos, version y todos los modulos a dist/scripts. Esta ruta debe conservarse porque el HTML referencia los scripts dentro de scripts. Cualquier cambio de estructura debe verificarse con una prueba de existencia de archivos antes de construir Android.

9. Orden de trabajo recomendado

Prioridad	Trabajo	Criterio de cierre
Alta	Corregir layout movil con zonas coordinadas, safe areas y espacio tactil inferior.	Inicio, Reportes y bienvenida sin solapamientos.
Alta	Resolver convivencia entre barra recurrente, FAB y bottom navigation.	Un solo carril por control y contenido siempre accesible.
Media	Revisar bottom sheet de bienvenida y formularios largos.	Scroll completo y boton final visible.
Media	Definir icono de notificacion y canal Android.	Icono de Saldo visible en panel.
Media	Mejorar mensajes expandibles de presupuesto y recurrentes.	Titulo, resumen y detalle accionable.
Baja	Agregar pruebas visuales por viewport y checklist de release.	Evitar regresiones entre APKs.

10. Criterios de aceptacion

- En un telefono pequeno, el contenido no queda oculto bajo la barra superior ni bajo la barra de navegacion del sistema.
- El boton de agregar gasto tiene un area tactil completa y no cubre ningun elemento interactivo.
- La barra o bloque de recurrentes no se superpone con el FAB ni con la navegacion inferior; si no hay pendientes, no ocupa espacio.
- La pantalla de bienvenida permite desplazarse hasta el boton final y ejecutarlo sin tocar la barra del sistema.
- Las notificaciones muestran el icono de Saldo, un titulo claro y un detalle completo al expandirse.
- Permisos denegados, canal desactivado y errores de API no bloquean la interfaz y quedan registrados.
- La compilacion vuelve a verificar que dist/scripts contiene todos los modulos antes de empaquetar Android.

11. Archivos y superficies a revisar

Superficie	Responsabilidad	Riesgo principal
styles.css	Layout responsive, posiciones fijas, safe areas, colores y estados visuales.	Solapamiento por valores independientes.
index.html	Estructura de vistas, navegacion, modales, barra recurrente y controles de notificacion.	Orden o contenedor incorrecto.
07-recurrentes.js	Render de barra, widget y confirmaciones.	Multiples representaciones con reglas diferentes.
14-interfaz.js	Cambio de vistas y render general.	Re-render que altera alturas o visibilidad.
15-eventos.js	Eventos de navegacion y formularios.	Controles visibles pero sin respuesta si falta un elemento.
17-notificaciones.js	Permisos, canal, envio y prueba Android.	Icono generico y cuerpo insuficiente.
src-tauri/src/lib.rs	Registro de plugins nativos.	API Android no disponible si el plugin no se registra.
capabilities/default.json	Permisos declarativos.	Logs o notificaciones bloqueados por permisos.
prepare-frontend.js	Empaquetado del frontend.	APK con HTML pero sin JavaScript si las rutas divergen.

Fin del documento. Este archivo describe el comportamiento y el plan de continuidad; no contiene código fuente.